

CONTENIDO

01 Resumen

03 Los ocho dominios

05 Los 28 huecos

07 Qué automatizar

09 Hoja de ruta

02 Arquitectura

04 Cómo funciona el acceso

06 Rutas fantasma

08 Módulos nuevos

Auditoría PAGADOR

Cómo está construido el sistema, los 28 huecos que tiene hoy, y qué hace falta para llevarlo de “funciona si nadie se equivoca” a una plataforma que se defiende sola.

Alcance backend + frontend + base de datos Base analizada 98.026 líneas

Entorno local aislado `pagador-local-isolated`

01 Resumen

98.026	2	0	0	15
líneas de código backend + frontend	archivos de prueba en todo el proyecto	validaciones en las 45 entidades	claves foráneas de 69 relaciones	ítems del menú que dan 404

Medido sobre el código fuente el 8 de septiembre de 2026, con la base local recién sincronizada (44 tablas, 54 rutas de menú, 4 roles).

PAGADOR es la plataforma administrativa interna del Gobierno Autónomo Departamental de Oruro. Cubre ocho dominios reales de una gobernación: recursos humanos, viáticos y comisiones, combustible y vehículos, apertura presupuestaria, contrataciones, archivo y actas, correspondencia (CITEs) y contabilidad. Es un sistema grande y ambicioso, y en el uso diario **funciona**.

El problema no es que falle: es que **no tiene ninguna barrera entre lo que un usuario escribe y lo que queda grabado**. La arquitectura por capas está dibujada —Entidad, Servicio, Repositorio, Result— pero cada capa está vacía por dentro. Las entidades no validan, la base de datos no exige integridad referencial, las operaciones de dinero no son atómicas, y los permisos que el sistema muestra en pantalla no se verifican en el servidor.

A eso se suman cuatro agujeros de seguridad explotables hoy mismo, entre ellos un endpoint que **borra tablas enteras sin pedir sesión** y una clave de firma de sesiones escrita dentro del código fuente.

Por qué “siempre le hacían mantenimiento”. No existen migraciones de base de datos: el único mecanismo de esquema es `sync({ force: true })`, que borra todas las tablas. En producción eso significa que cada cambio de columna se hizo a mano, sin historial y sin poder revertirse. Sumado a 2 pruebas automatizadas en 98.000 líneas, cualquier cambio es cirugía a ciegas.

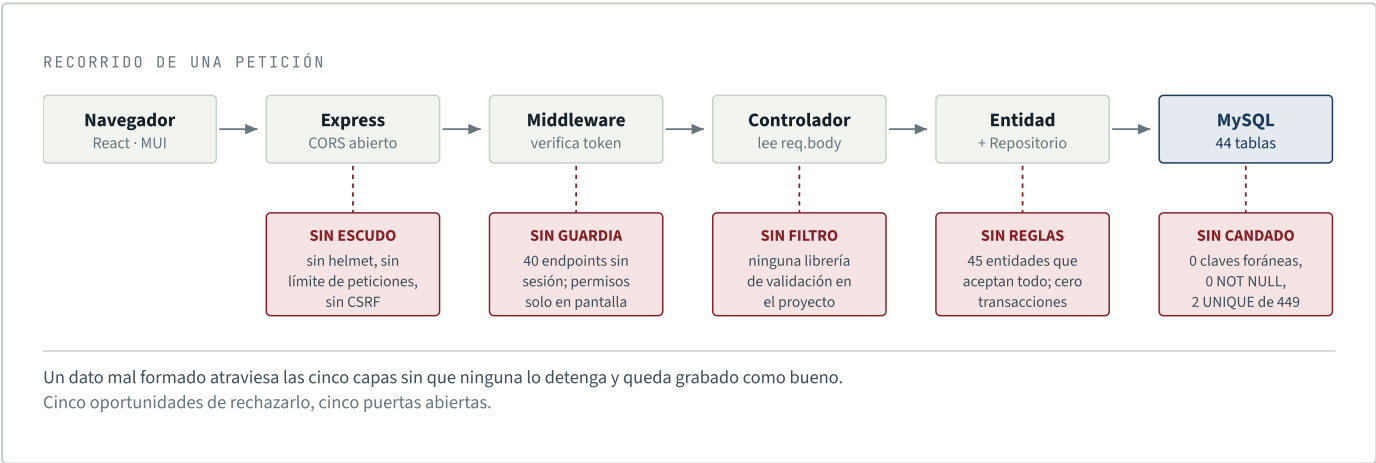
Ese es el costo real: no es que el sistema esté “mal hecho”, es que **no tiene red de seguridad**, y por eso cada ajuste exige una persona vigilándolo.

02 Arquitectura

El backend es Node + Express + Sequelize sobre MySQL 8, organizado por módulos. Cada entidad de negocio repite la misma estructura de cinco archivos:

CAPA	ARCHIVO	QUÉ DEBERÍA HACER	QUÉ HACE HOY
Ruta	*ViewControllerRoutes.ts	Declarar endpoints y guardias	40 sin guardia
Controlador	*ViewController.ts	Leer req.body y validarlo	no valida
Vista	*View.ts	Componer la respuesta	cumple
Servicio	*Service.ts	Reglas de negocio	casi vacío
Entidad	*Entity.ts	Invariantes del dominio	0 de 45
Repositorio	*RepositorySQL.ts	Persistencia y transacciones	sin transacción

EL CAMINO DE UN DATO, Y DÓNDE SE PIERDE EL CONTROL



Cada capa existe y tiene su archivo; ninguna ejerce el control que le corresponde.

03 Los ocho dominios

41 módulos de API, 44 tablas y 56 pantallas repartidos así:

Sistema /users · /roles · /rutas · /bitacora · /setting · /storage Usuarios, roles, catálogo de pantallas y auditoría. Toda petición autenticada se registra automáticamente. usuario · role · ruta · acceso · bitacora	Recursos Humanos /personal · /area · /cargo · /vacacion · /memorandum_rrhh Planilla, estructura organizativa y vacaciones con flujo de aprobación en dos pasos: jefe, luego RR.HH. personal · area · cargo · vacacion · memorandum_rrhh · detalle_destino_rrhh
---	--

Viáticos /memorandum · /viatico · /descargo · /escala · /apertura_viatico El dominio más completo: designación de comisión, cálculo de pasajes y viático según escala y destino, pago y rendición. apertura_viatico · memorandum · escala · escala_destino · viatico · descargo · detalle_destino · informe_comision · informe_general · vehiculo_publico	Bienes y Combustible /vale · /vehiculo · /destino · /asignacion · /bitacora_viaje Vales de combustible con estados, parque automotor, destinos tarifados y bitácora de recorridos. vale · vehiculo · destino · asignacion · bitacora_viaje · bitacora_detalle
Apertura presupuestaria /apertura_general · /objeto_gasto · /historial_apertura Presupuesto por apertura programática y partida: inicial, modificaciones, vigente, pagado y saldo por ejecutar. apertura_general · objeto_gasto · historial_detalle_ap · historial_gasto_ap	Contrataciones /proceso · /actividad · /general Procesos con CUCE, modalidad y hoja de ruta, con actividades y plazos asociados. Tenía enlace al sistema externo SIGAPO, hoy inactivo. proceso · actividad · general
Archivo /documento · /acta · /acta_recepcion Gestión documental y actas de recepción por área, con código, adjuntos y marca de sellado. documento · acta · acta_recepcion · acta_recepcion_detalle	Correspondencia y Contabilidad /cites · /cuenta · /deuda Registro de CITEs con área solicitante, destino, referencia y plazo; y el módulo contable de cuentas y deudas. cites · tipo_cites · cuenta · deuda · seguimiento · historial

PATRONES COMUNES A TODAS LAS TABLAS

- `id` es siempre UUID de 36 caracteres, nunca autoincremental.
- Las relaciones se nombran `fid_algo` (`fid_area` , `fid_usuario` , `fid_personal`) y casi todo termina conectando con RR.HH.
- El borrado es lógico: `fecha_eliminacion` se llena en vez de borrar la fila. Toda consulta manual debe filtrar `WHERE fecha_eliminacion IS NULL` .
- Los montos usan `DOUBLE` , no `DECIMAL` — aceptable para reportes, arriesgado para cuadros exactos al centavo.
- Casi todo tiene una máquina de estados: `PENDIENTE` → `EN_PROCESO` / `APROBADO` → `ATENDIDO` / `PAGADO` , con salidas a `RECHAZADO` o `ANULADO` .

04 Cómo funciona el acceso

Tres tablas gobiernan todo el sistema: `usuario` apunta a `role`, y `role` declara qué entradas de `ruta` puede ver.

PASO	QUÉ OCURRE	DÓNDE
1	El usuario envía correo y contraseña	POST <code>/api/users/login</code>
2	El servidor busca por la columna <code>username</code> , no por <code>email</code> , y compara el hash bcrypt	<code>UsuarioService.findByEmail</code>
3	Devuelve un token JWT de 7 días con el tipo de rol dentro	<code>base/utils/jwt.ts</code>
4	El frontend pide la sesión y recibe <code>{ roles, permisos, modulos[] }</code>	GET <code>/api/users/session-info</code>
5	El menú se dibuja cruzando <code>role.modulos</code> con la tabla <code>ruta</code>	<code>UsuarioView.getSessionInfo</code>

El punto crítico está en el paso 5. Los permisos — `create`, `edit`, `remove`, `approve` — y la lista de módulos solo deciden *qué se dibuja en pantalla*. El servidor nunca los consulta antes de escribir. Un usuario con sesión válida puede llamar por su cuenta cualquier endpoint de cualquier módulo, vea o no ese módulo en su menú.

El único control real que existe es `checkIfSuperAdmin`, presente en 17 de los 349 puntos protegidos.

Los cuatro roles sembrados son `superadministrador`, `administrador`, `tecnico` y `jefe`. El identificador lógico vive en `role.tipo`; `role.nombre` es solo la etiqueta visible.

05 Los 28 huecos

Cada hallazgo se verificó leyendo el código o probándolo contra el entorno local. El orden es por severidad dentro de cada grupo.

SEGURIDAD — 11 HALLAZGOS

SEC-01 Un endpoint sin sesión borra tablas completas

CRÍTICO

`GET /api/setting/purgar_all` no lleva ningún middleware de autenticación y termina en `destroy({ where: {}, truncate:true, force:true })`, que vacía la tabla físicamente saltándose el borrado lógico.

DÓNDE `views/system/setting/controller/SettingViewControllerRoutes.ts:23` · `base/infra/_sql/BaseSQLRepository.ts`

RIESGO Al ser `GET`, basta que alguien abra la URL. Un rastreador, una precarga del navegador o un enlace pegado por error destruyen datos sin vuelta atrás.

ARREGLO Eliminar la ruta. Si se necesita, convertirla en `POST` con `checkIfSuperAdmin`, confirmación explícita y registro en bitácora.

SEC-02 La clave que firma las sesiones está escrita en el código

CRÍTICO

El secreto JWT tiene un valor de reserva literal dentro del repositorio: `process.env.JWT_SECRET || "gobierno_autonomo..."`. Si el servidor arranca sin la variable de entorno, usa ese valor.

DÓNDE `backentrega/src/base/utils/jwt.ts:4`

RIESGO Cualquiera con acceso al código fuente puede fabricar un token de superadministrador válido y entrar como quien quiera.

ARREGLO Quitar el valor de reserva y hacer que el proceso **no arranque** si falta `JWT_SECRET`. Rotar el secreto en producción invalida las sesiones actuales, que es justamente lo que se busca.

SEC-03 Cerca de 40 endpoints de lectura no piden sesión

CRÍTICO

Presupuesto, contrataciones, escalas de viático y estructura de personal se sirven sin `checkIfAuthenticated`. Entre ellos `/api/apertura_general`, `/api/procesos`, `/api/cites_all_documentos`, `/api/areas`, `/api/cargos`, `/api/vehiculos`, `/api/apertura_viatico`, `/api/escala` y `/api/historial_apertura`.

DÓNDE múltiples `*ViewControllerRoutes.ts` en `views/admin` y `views/rrhh`

RIESGO Quien alcance la API lee información presupuestaria y de contrataciones sin credenciales.

ARREGLO Invertir el criterio: aplicar el guardia a nivel de router en `Api.ts` y declarar explícitamente las poquísimas rutas públicas.

SEC-04 Los archivos subidos se descargan sin autenticación

CRÍTICO

`GET /api/storage/file/:filename` es la única ruta del módulo sin guardia, y los archivos conservan su nombre original, así que son adivinables.

DÓNDE `views/system/storage/controller/StorageViewControllerRoutes.ts`

RIESGO Actas, memorándums y respaldos de descargo quedan expuestos a quien acierte el nombre.

ARREGLO Exigir sesión, guardar con nombre aleatorio y verificar que quien pide el archivo tenga derecho al registro que lo referencia.

SEC-05 Se acepta cualquier archivo, de cualquier tipo y tamaño

ALTO

`uploadFile` no comprueba tipo MIME, extensión ni peso, y guarda con el nombre que envía el cliente. Dos archivos con el mismo nombre se pisan en silencio.

DÓNDE `views/system/storage/controller/StorageViewController.ts:15`

ARREGLO Lista blanca de tipos, límite de tamaño en Busboy y nombre generado por el servidor.

SEC-06 Un archivo subido puede ejecutar código en el navegador

ALTO

`getFile` responde con `Content-Disposition: inline` y un `Content-Type` deducido del nombre del archivo. Un `.html` o `.svg` subido se ejecuta dentro del origen de la API.

DÓNDE `views/system/storage/controller/StorageViewController.ts:117`

RIESGO Robo de sesiones de otros usuarios (XSS almacenado) desde un archivo que parece un adjunto normal.

ARREGLO Servir siempre como `attachment`, forzar `Content-Type` a partir de la lista blanca y añadir `X-Content-Type-Options: nosniff`.

SEC-07 Cualquier usuario con sesión borra cualquier archivo

ALTO

`DELETE /api/storage/upload` recibe solo un `fileName` y lo elimina. No comprueba quién subió el archivo ni a qué registro pertenece.

ARREGLO Verificar propiedad contra el registro que referencia el archivo antes de borrar.

SEC-08 Los permisos del rol no se verifican en el servidor

ALTO

Los indicadores `create`, `edit`, `remove` y `approve` aparecen en apenas 6 lugares del código, todos dentro de archivos `*View.ts` y solo para decidir qué mostrar. Ningún endpoint de escritura los consulta.

RIESGO Un técnico sin permiso de aprobación puede aprobar llamando al endpoint directamente.

ARREGLO Middleware `requierePermiso('approve')` aplicado en las rutas de escritura, leyendo el rol desde la base y no desde el token.

SEC-09 El login revela qué correos existen

MEDIO

Ante un usuario inexistente responde `500` con el texto `"Usuario no encontrado con ese email: X"`; ante una contraseña incorrecta responde otro mensaje. La diferencia permite enumerar cuentas.

ARREGLO Responder siempre `401` con un mensaje único, como “credenciales inválidas”.

SEC-10 Las sesiones duran 7 días y no se pueden revocar

MEDIO

`expiresIn: "7d"` sin token de refresco ni lista de revocación. Cerrar sesión solo borra el token del navegador; el token sigue siendo válido una semana.

ARREGLO Token de acceso corto (15–30 min) más token de refresco revocable, y una lista de revocación consultada al validar.

SEC-11 Sin cabeceras de seguridad, sin límite de peticiones, CORS abierto

MEDIO

No hay `helmet` ni limitador de intentos; `cors()` se llama sin opciones, aceptando cualquier origen. La configuración de base de datos fija además `ssl.rejectUnauthorized: false`, que acepta cualquier certificado.

RIESGO Sin límite de intentos, la contraseña de cualquier cuenta puede probarse por fuerza bruta indefinidamente.

INTEGRIDAD DE DATOS — 5 HALLAZGOS

INT-01 Ninguna de las 45 entidades valida nada

CRÍTICO

Cada `XEntity.create()` es literalmente `return Result.ok(new XEntity(props, id))`. Cero llamadas a `Result.fail` en las 45 entidades. Tampoco hay librería de validación (joi, zod, express-validator) entre las dependencias.

CONSECUENCIA Se puede grabar un viático con `total_viatico = -5000`, una vacación cuya fecha de fin es anterior a la de inicio, o un personal sin CI. Nadie pregunta nada antes de guardar.

ARREGLO Un esquema por entidad validado en el controlador, antes de llegar al servicio. Es el cambio de mayor retorno de toda la lista.

INT-02 La base de datos no tiene una sola clave foránea

CRÍTICO

Las 69 asociaciones del proyecto declaran `constraints: false`. MySQL no rechaza un `viatico` cuyo `fid_memorandum` apunta a un memorándum inexistente, ni impide borrar un personal con vales asociados.

CONSECUENCIA Registros huérfanos en cualquier tabla, y reportes que suman sobre datos que ya no tienen padre.

ARREGLO Activarlas por dominio, empezando por viáticos y presupuesto, tras una limpieza de huérfanos existentes.

INT-03 Las operaciones de dinero no son atómicas

ALTO

Cero usos de `sequelize.transaction` en 44.378 líneas de backend. Un viático descuenta el presupuesto, crea el recibo y escribe el historial de gasto como tres escrituras independientes.

CONSECUENCIA Si el proceso se corta entre la primera y la tercera escritura, el presupuesto queda descontado sin recibo, o al revés. No hay reversión.

ARREGLO Envolver cada flujo de dinero en una transacción y propagarla hasta el repositorio.

INT-04 El esquema no exige nada: ni obligatorios ni únicos

ALTO

De 449 definiciones de columna, `allowNull: false` aparece **0 veces** y `unique` solo **2 veces**.

CONSECUENCIA Nada impide dos usuarios con el mismo correo, dos vales con el mismo código o dos roles con el mismo tipo.

ARREGLO Índices únicos en `usuario.email`, `usuario.username`, `role.tipo`, `vale.cod_vale`, `acta_recepcion.cod_acta` y `cites.cite_completo`, más `NOT NULL` en los campos que el negocio ya considera obligatorios.

INT-05 Tipo de columna inválido en `historial_detalle_ap.gasto`

CORREGIDO

El campo usaba `Sequelize.NUMBER()`, que es la clase abstracta base de los tipos numéricos: genera el literal `NUMBER` como DDL, inválido en todos los dialectos. Impedía crear la tabla.

DÓNDE `core/admin/apertura/historial_apertura/infra/_sql/HistorialAperturaSQLModel.ts:18`

ESTADO Corregido a `DOUBLE()`, coherente con los campos monetarios del modelo hermano `historial_gasto_ap`. Es un síntoma de MAN-02: sin pruebas, un tipo roto sobrevivió en el código.

FUNCIONALIDAD ROTA — 7 HALLAZGOS

FUN-01 El control de duplicados no hace nada

ALTO

`repo.exists()` consulta `this.entities`, un arreglo en memoria que nunca se llena desde la base. Siempre devuelve `false`, así que la comprobación “¿ya existe?” de `BaseService.create` es decorativa.

DÓNDE `base/infra/_sql/BaseSQLRepository.ts`

FUN-02 `findAll()` es un esqueleto que devuelve lista vacía

ALTO

El método paginado devuelve `{ rows: this.entities, count: this.entities.length }` sobre el mismo arreglo vacío. Toda pantalla que lo use lista cero registros y muestra un total incorrecto.

FUN-03 Cada inicio de sesión recorre la tabla completa de usuarios

ALTO

`getAll()` sin argumentos no aplica `LIMIT`: trae la tabla entera a memoria. `UsuarioService.findByEmail` la usa y luego filtra en JavaScript con `.find()`.

ARREGLO Sustituir por `findOne({ where: { username } })` con índice. Es una línea y elimina el cuello de botella.

FUN-04 15 de 54 entradas del menú llevan a una página inexistente

ALTO

La tabla `ruta` ofrece módulos que el frontend nunca registró. Detalle completo en la sección siguiente.

FUN-05 La búsqueda solo mira las primeras 5 filas

MEDIO

Cuando se envía un objeto de consulta, el límite por defecto es `DEFAULT_QUERY_LIMIT = 5`. El filtro de texto `?q=` se aplica **después**, en JavaScript, sobre esas 5 filas ya recortadas.

CONSECUENCIA Buscar un personal por apellido en una planilla de cientos casi siempre devuelve “sin resultados”.

FUN-06 Dos rutas del menú apuntan a direcciones que la aplicación no usa

MEDIO

El menú envía a `/rrhh/vale_admin` y `/bienes/apertura`, pero la aplicación registra `/bienes/vale_admin` y sirve `Apertura.tsx` bajo `/bienes/asignacion`.

FUN-07 Mensaje de error vacío por copiar y pegar

BAJO

`BaseService.create` devuelve `entity.error` cuando quien falló fue `exists`; debería devolver `exists.error`. El usuario recibe un error sin texto.

MANTENIBILIDAD — 5 HALLAZGOS

MAN-01 No existen migraciones de base de datos

CRÍTICO

`.sequelizerc` apunta a `sequelize/migrations`, una carpeta que no existe. El único mecanismo de esquema es `sync({ force: true })`, que borra todas las tablas antes de recrearlas.

CONSECUENCIA En producción cada cambio de columna se hizo con `ALTER TABLE` a mano, sin historial, sin poder revertir y sin garantía de que el esquema real coincida con los modelos del código.

ARREGLO Generar una migración inicial desde el esquema actual y prohibir `sync({force:true})` fuera de desarrollo. Es la condición previa para poder evolucionar el sistema con seguridad.

MAN-02 Dos pruebas automatizadas en 98.000 líneas

ALTO

Solo existen `ProcesoTest.test.ts` en el backend y el `App.test.tsx` que Create React App genera por defecto.

CONSECUENCIA Ningún cambio se puede verificar sin abrir la aplicación y probarlo a mano. Los errores llegan a producción porque no hay nada que los detenga antes.

MAN-03 Sin integración continua

ALTO

No hay ningún flujo automático que compile, pruebe o revise el código antes de desplegar. El despliegue es un `caprover deploy` manual.

MAN-04 Cero automatización de fondo

MEDIO

No hay tareas programadas, colas ni procesos en segundo plano en todo el backend. Todo cambio de estado —marcar un proceso atrasado, avisar de un descargo vencido, recalcular un saldo— depende de que una persona entre y haga clic.

OPORTUNIDAD Es el punto con más recorrido del sistema. Ver la sección 07.

MAN-05 Deuda técnica visible y confusión de nombres

BAJO

Bloques comentados extensos por todo el código, `this.sequelize` y `this._sequelize` mezclados en `Database.ts`, `operatorsAliases: 1` (opción retirada en Sequelize 5) y un login “por email” que en realidad compara contra `username` — la trampa que hace fallar cualquier cuenta nueva creada con esos dos campos distintos.

06 Rutas fantasma

El menú se construye desde la tabla `ruta`, pero el frontend registra sus pantallas en `App.tsx`. Quince entradas existen en la primera y no en la segunda: aparecen en el menú y llevan a una página en blanco.

ENTRADA DEL MENÚ	RUTA QUE ABRE	ESTADO	QUÉ FALTA
Horario	/rrhh/horario	404	Bloque completo de control de asistencia: definición de horarios, rangos, marcate y designación por funcionario. No existe ni backend ni frontend.
Horario Rango	/rrhh/horario_rango	404	
Registro Horario	/rrhh/registro_horario	404	
Designación Horario	/rrhh/designacion_horario	404	
Configuración Horario	/rrhh/config_horario	404	
Permiso	/rrhh/permiso	404	Solicitud y designación de permisos del funcionario.
Designar Permiso	/rrhh/designacion_permiso	404	
Ficha	/rrhh/ficha	404	Ficha del funcionario en modo autoservicio y en modo administrador.
Ficha Admin	/rrhh/ficha_admin	404	
Declaración	/rrhh/declaracion	404	Declaración jurada del funcionario.
Notificación	/dashboard/notificacion	404	Centro de notificaciones. El menú lo promete pero no hay backend ni tabla.
Sigapo	/gabi/sigapo	404	Consulta al sistema externo de correspondencia. El backend existe pero apunta a URLs de ejemplo.

ENTRADA DEL MENÚ	RUTA QUE ABRE	ESTADO	QUÉ FALTA
Excel	/viatico/archivo_excel	404	Exportación masiva de viáticos a Excel.
Vale Admin	/rrhh/vale_admin	404	La pantalla sí existe , pero registrada en <code>/bienes/vale_admin</code> . Corregir la ruta en la tabla es un <code>UPDATE</code> .
Apertura	/bienes/apertura	404	La pantalla sí existe , pero se sirve en <code>/bienes/asignacion</code> . También se arregla con un <code>UPDATE</code> .
Solicitud Vacación	/rrhh/solicitud_vacacion	parcial	Solo está registrada la variante con identificador (<code>/:id</code>); el listado no existe.

Dos de las quince se arreglan hoy mismo. “Vale Admin” y “Apertura” tienen la pantalla construida y funcionando; solo está mal la dirección guardada en la tabla `ruta`. Son dos sentencias `UPDATE`.

Las trece restantes son funcionalidad que nunca se construyó. Lo más honesto mientras tanto es **ocultarlas del menú**, para que nadie las descubra a la mala.

07 Qué automatizar

Hoy el sistema es un registrador pasivo: guarda lo que la gente escribe y no hace nada por su cuenta. Los diez puntos siguientes ya tienen los datos necesarios en la base — lo único que falta es el disparador.

01 Vencimientos y estados atrasados

Las tablas `proceso`, `actividad` y `cites` ya tienen estados `ATRASADO` y `ATENDIDO_CON_RETRASO`, campos `dias` y `fecha_cierre`. Una tarea nocturna que recorra plazos, cambie el estado y avise al responsable.

Hoy: alguien revisa y marca a mano. La función `obtenerFeriadosBolivia()` para calcular días hábiles ya existe y no se usa para esto.

IMPACTO ALTO

02 Conciliación presupuestaria nocturna

Verificar cada noche que `apertura_general.pagado` coincida con la suma de `historial_gasto_ap`, y que `presupuesto_vigente = inicial + mod_aprobada`. Reportar cualquier descuadre.

Hoy: los saldos se recalculan dentro de la petición, sin transacción (INT-03). Un corte a media operación deja el presupuesto mal y nadie se entera.

IMPACTO ALTO

03 Numeración correlativa automática

`cod_vale`, `cod_acta`, `cite_completo` y `nume_recibo` deberían generarse en el servidor, por gestión y por área, dentro de la misma transacción que crea el registro.

Hoy: se arman a mano y no hay índice único (INT-04), así que dos personas trabajando a la vez pueden generar el mismo número.

IMPACTO ALTO

04 Control de descargos vencidos

Detectar viáticos cuyo plazo de rendición pasó, notificar al funcionario y a su jefe, y bloquear nuevas solicitudes a quien tenga descargos pendientes.

Hoy: nada impide que un funcionario acumule viáticos sin rendir ninguno.

IMPACTO ALTO

05 Notificaciones por correo y en la plataforma

Avisos automáticos en los momentos que ya define el flujo: memorándum designado, viático listo para cobro, vacación aprobada por el jefe, proceso próximo a vencer.

Hoy: el menú tiene una entrada “Notificación” que no lleva a ninguna parte (FUN-04), y `nodemailer` está instalado pero solo se usa para restablecer contraseñas.

IMPACTO ALTO

06 Vacaciones devengadas por antigüedad

La tabla `personal` ya guarda `fecha_ingreso`, `anhos_antigüedad_gador` y `anhos_antigüedad_cas`. Calcular automáticamente los días que corresponden según escala y descontar los aprobados.

Hoy: el saldo de vacaciones se lleva por fuera del sistema.

IMPACTO MEDIO

07 Detección de consumo anómalo de combustible

Cruzar los litros de `vale` contra la distancia registrada en `bitacora_viaje` por vehículo y marcar los rendimientos fuera de rango histórico.

Hoy: ambas tablas se llenan pero nunca se comparan. Es control interno de alto valor y bajo costo.

IMPACTO ALTO

08 Reportes programados

Ya existen doce generadores de reportes en `tools/*Report.ts`. Falta el calendario: enviar el día 1 de cada mes el consolidado de ejecución presupuestaria y de vales a cada jefe de área.

Hoy: cada reporte se genera solo si alguien entra y lo pide.

IMPACTO MEDIO

09 Respaldos verificados

`mysqldump` diario con retención, y —lo que casi nadie hace— una restauración de prueba semanal que confirme que el respaldo sirve.

Hoy: no existe ninguna rutina de respaldo en el proyecto.

IMPACTO ALTO

10 Contraste con sistemas externos

Los procesos guardan `cuce`, el identificador único de contratación estatal. Contrastarlo periódicamente contra la fuente oficial detecta desfases entre lo registrado internamente y lo publicado.

Hoy: la integración SIGAPO existe en código pero apunta a direcciones de ejemplo y nunca funcionó.

IMPACTO MEDIO

Todo esto necesita una sola pieza de infraestructura: un proceso planificador con registro de ejecución y reintentos, separado del servidor web. Con esa pieza en pie, cada automatización de la lista son entre 50 y 200 líneas, porque los datos ya están.

08 Módulos nuevos

Antes de agregar nada conviene terminar lo prometido: las trece pantallas fantasma de la sección 06, sobre todo el bloque de asistencia y horarios, que es el más grande y el que más se nota que falta. Hecho eso, estas son las incorporaciones que mejor encajan con lo que el sistema ya tiene.

MÓDULO	QUÉ RESUELVE	SOBRE QUÉ SE APOYA
Bitácora consultable	La tabla <code>bitacora</code> ya registra usuario, ruta, método e IP de cada petición, pero no hay pantalla para consultarla. Una vista con filtros convierte datos muertos en trazabilidad real.	<code>bitacora</code> (ya se llena)
Portal del funcionario	Autoservicio: mis viáticos, mis vales, mis permisos, mis vacaciones, mis descargos pendientes. Descarga la mayor parte de las consultas que hoy llegan por mesa de entrada.	<code>viatico</code> · <code>vale</code> · <code>vacacion</code> · <code>descargo</code> · <code>personal</code>

MÓDULO	QUÉ RESUELVE	SOBRE QUÉ SE APOYA
Tablero de indicadores	Ejecución presupuestaria por área y partida, procesos por estado, consumo de combustible, viáticos pendientes de descargo. Hoy el panel solo saluda y muestra la fecha.	<code>apertura_general</code> · <code>proceso</code> · <code>vale</code> · <code>viatico</code>
Hoja de ruta digital	Seguimiento de un documento entre áreas de punta a punta, uniendo lo que hoy está separado en tres módulos distintos.	<code>cites</code> · <code>documento</code> · <code>acta_recepcion</code>
Firma y sellado de actas	<code>acta_recepcion</code> ya tiene un campo <code>sellado</code> booleano sin uso real. Convertirlo en un sellado con huella criptográfica y fecha da valor probatorio a los documentos.	<code>acta_recepcion</code> (campo ya existe)
Contratos y garantías	Vencimiento de boletas de garantía y plazos contractuales con aviso anticipado. Es la continuación natural del módulo de contrataciones.	<code>proceso</code> · <code>actividad</code>
Activos fijos y almacén	El único dominio administrativo clásico que falta por completo: inventario, asignación de bienes a funcionarios y bajas.	<code>nuevo</code> · <code>se apoya en personal y area</code>

09 Hoja de ruta

El orden importa más que la velocidad. Automatizar sobre cimientos flojos multiplica los errores en vez de reducirlos: si el sistema acepta datos inválidos, una tarea automática los propaga a mil registros en una noche.

FASE 0 Contención

≈ 1 semana

Cuatro cosas que se arreglan rápido y que, hasta que no estén hechas, vuelven irrelevante todo lo demás.

- Eliminar o proteger `purgar_all` (SEC-01)
- Sacar el secreto JWT del código y exigir la variable de entorno (SEC-02)
- Aplicar el guardia de sesión a nivel de router y declarar las excepciones (SEC-03)
- Cerrar la descarga pública de archivos (SEC-04)
- Ocultar del menú las trece rutas sin pantalla y corregir las dos mal escritas (FUN-04, FUN-06)

FASE 1 Cimientos

≈ 3-4 semanas

Que el sistema deje de aceptar cualquier cosa. Es la fase que cambia la naturaleza del proyecto.

- Capa de validación por esquema en los controladores (INT-01)
- Migración inicial y prohibición de `sync({force:true})` fuera de desarrollo (MAN-01)
- Claves foráneas, `NOT NULL` e índices únicos, por dominio (INT-02, INT-04)
- Transacciones en viáticos, presupuesto y vales (INT-03)
- Verificación de permisos en el servidor (SEC-08)
- Endurecer subida y descarga de archivos (SEC-05, SEC-06, SEC-07)

FASE 2 Confianza

≈ 4-6 semanas

Que un cambio se pueda hacer sin miedo. Sin esta fase, la anterior se degrada sola con el tiempo.

- Pruebas sobre los flujos de dinero: cálculo de viático, descuento de presupuesto, emisión de vale
- Integración continua que compile, pruebe y revise antes de desplegar (MAN-03)
- Corregir `exists`, `findAll`, `findByEmail` y la búsqueda (FUN-01, FUN-02, FUN-03, FUN-05)
- Sesiones cortas con token de refresco revocable (SEC-10)
- Cabeceras de seguridad, límite de intentos y CORS acotado (SEC-11)

FASE 3 Automatización

≈ 6-8 semanas

Aquí el sistema deja de ser un registrador y empieza a trabajar solo. Es lo que más se va a notar en el día a día.

- Proceso planificador con registro de ejecución y reintentos (MAN-04)
- Respalos verificados, primero de todo lo demás
- Vencimientos, notificaciones y numeración correlativa
- Conciliación presupuestaria y control de descargos vencidos
- Detección de consumo anómalo de combustible y reportes programados

FASE 4 Ampliación

continuo

Con cimientos, pruebas y automatización, agregar módulos deja de ser riesgoso.

- Completar el bloque de asistencia y horarios
- Bitácora consultable y tablero de indicadores
- Portal del funcionario
- Hoja de ruta digital, firma de actas, contratos y garantías, activos fijos

Si solo se pudiera hacer una cosa, sería la Fase 1: la validación por esquema más las restricciones en la base. Es lo que convierte “el sistema guarda lo que sea” en “el sistema solo guarda datos válidos”, y a partir de ahí todo lo demás —automatizar, ampliar, confiar en los reportes— se vuelve posible.

Revisión realizada sobre el código fuente y un entorno local completamente aislado (`pagador-local-isolated`): base de datos ficticia, red Docker interna sin salida a internet y ninguna credencial, dirección ni dato institucional real. Ningún hallazgo se verificó contra sistemas en producción.